



THE UNIVERSITY *of* EDINBURGH

Edinburgh Research Explorer

PyTorch Geometric Temporal: Spatiotemporal Signal Processing with Neural Machine Learning Models

Citation for published version:

Rozemberczki, B, Scherer, P, He, Y, Panagopoulos, G, Riedel, A, Astefanoaei, M, Kiss, O, Beres, F, López, G, Collignon, N & Sarkar, R 2021, PyTorch Geometric Temporal: Spatiotemporal Signal Processing with Neural Machine Learning Models. in *CIKM '21: Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. Association for Computing Machinery (ACM), pp. 4564–4573, 30th ACM International Conference on Information and Knowledge Management, Gold Coast, Queensland, Australia, 1/11/21. <https://doi.org/10.1145/3459637.3482014>

Digital Object Identifier (DOI):

[10.1145/3459637.3482014](https://doi.org/10.1145/3459637.3482014)

Link:

[Link to publication record in Edinburgh Research Explorer](#)

Document Version:

Peer reviewed version

Published In:

CIKM '21: Proceedings of the 30th ACM International Conference on Information & Knowledge Management

General rights

Copyright for the publications made accessible via the Edinburgh Research Explorer is retained by the author(s) and / or other copyright owners and it is a condition of accessing these publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy

The University of Edinburgh has made every reasonable effort to ensure that Edinburgh Research Explorer content complies with UK legislation. If you believe that the public display of this file breaches copyright please contact openaccess@ed.ac.uk providing details, and we will remove access to the work immediately and investigate your claim.



PyTorch Geometric Temporal: Spatiotemporal Signal Processing with Neural Machine Learning Models

Benedek Rozemberczki*

AstraZeneca
United Kingdom
benedek.rozemberczki@astrazeneca.com

Paul Scherer
University of Cambridge
United Kingdom
pms69@cam.ac.uk

Yixuan He
University of Oxford
United Kingdom
yixuan.he@stats.ox.ac.uk

George Panagopoulos
École Polytechnique
France
george.panagopoulos@polytechnique.edu

Alexander Riedel
Ernst-Abbe University for Applied
Sciences
Germany
alexander.riedel@eah-jena.de

Maria Astefanoaei
IT University of Copenhagen
Denmark
msia@itu.dk

Oliver Kiss
Central European University
Hungary
kiss_oliver@phd.ceu.edu

Ferenc Beres
ELKH SZTAKI
Hungary
beres@sztaki.hu

Guzmán López
Tryolabs
Uruguay
guzman@tryolabs.com

Nicolas Collignon
Pedal Me
United Kingdom
nicolas@pedalme.co.uk

Rik Sarkar
The University of Edinburgh
United Kingdom
rsarkar@inf.ed.ac.uk

ABSTRACT

We present PyTorch Geometric Temporal a deep learning framework combining state-of-the-art machine learning algorithms for neural spatiotemporal signal processing. The main goal of the library is to make temporal geometric deep learning available for researchers and machine learning practitioners in a unified easy-to-use framework. PyTorch Geometric Temporal was created with foundations on existing libraries in the PyTorch eco-system, streamlined neural network layer definitions, temporal snapshot generators for batching, and integrated benchmark datasets. These features are illustrated with a tutorial-like case study. Experiments demonstrate the predictive performance of the models implemented in the library on real world problems such as epidemiological forecasting, ride-hail demand prediction and web-traffic management. Our sensitivity analysis of runtime shows that the framework can potentially operate on web-scale datasets with rich temporal features and spatial structure.

*The project started when the author was a doctoral student of the Center for Doctoral Training in Data Science at The University of Edinburgh.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM'21, 1-5 November 2021, Online

© 2021 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

ACM Reference Format:

Benedek Rozemberczki, Paul Scherer, Yixuan He, George Panagopoulos, Alexander Riedel, Maria Astefanoaei, Oliver Kiss, Ferenc Beres, Guzmán López, Nicolas Collignon, and Rik Sarkar. 2021. PyTorch Geometric Temporal: Spatiotemporal Signal Processing with Neural Machine Learning Models. In *CIKM'21: ACM International Conference on Information and Knowledge Management, 1-5 November 2021, Online*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Deep learning on static graph structured data has seen an unprecedented success in various business and scientific application domains. Neural network layers which operate on graph data can serve as building blocks of document labeling, fraud detection, traffic forecasting and cheminformatics systems [7, 45–47, 63]. This emergence and the wide spread adaptation of geometric deep learning was made possible by open-source machine learning libraries. The high quality, breadth, user oriented nature and availability of specialized deep learning libraries [13, 15, 46, 67] were all contributing factors to the practical success and large-scale deployment of graph machine learning systems. At the same time the existing geometric deep learning frameworks operate on graphs which have a fixed topology and it is also assumed that the node features and labels are static. Besides limiting assumptions about the input data, these off-the-shelf libraries are not designed to operate on spatiotemporal data.

Present work. We propose PyTorch Geometric Temporal, an open-source Python library for spatiotemporal machine learning. We designed PyTorch Geometric Temporal with a simple and consistent API inspired by the software architecture of existing widely

used geometric deep learning libraries from the PyTorch ecosystem [15, 40]. Our framework was built by applying simple design principles consistently. The framework reuses existing neural network layers in a modular manner, models have a limited number of public methods and hyperparameters can be inspected. Spatiotemporal signal iterators ingest data memory efficiently in widely used scientific computing formats and return those in a PyTorch compatible format. The design principles in combination with the test coverage, documentation, practical tutorials, continuous integration, package indexing and frequent releases make the framework an end-user friendly spatiotemporal machine learning system.

The experimental evaluation of the framework entails node level regression tasks on datasets released exclusively with the framework. Specifically, we compare the predictive performance of spatiotemporal graph neural networks on epidemiological forecasting, demand planning, web traffic management and social media interaction prediction tasks. Synthetic experiments show that with the right batching strategy PyTorch Geometric Temporal is highly scalable and benefits from GPU accelerated computing.

Our contributions. The main contributions of our work can be summarized as:

- We publicly release *PyTorch Geometric Temporal* the first deep learning library for parametric spatiotemporal machine learning models.
- We provide data loaders and iterators with *PyTorch Geometric Temporal* which can handle spatiotemporal datasets.
- We release new spatiotemporal benchmark datasets from the renewable energy production, epidemiological reporting, goods delivery and web traffic forecasting domains.
- We evaluate the spatiotemporal forecasting capabilities of the neural and parametric machine learning models available in *PyTorch Geometric Temporal* on real world datasets.

The remainder of the paper has the following structure. In Section 2 we overview important preliminaries and the related work about temporal and geometric deep learning and the characteristics of related open-source machine learning software. The main design principles of *PyTorch Geometric Temporal* are discussed in Section 3 with a practical example. We demonstrate the forecasting capabilities of the framework in Section 4 where we also evaluate the scalability of the library on various commodity hardware. We conclude in Section 5 where we summarize the results. The source code of *PyTorch Geometric Temporal* is publicly available at https://github.com/benedekrozemberczki/pytorch_geometric_temporal; the Python package can be installed via the *Python Package Index*. Detailed documentation is accessible at <https://pytorch-geometric-temporal.readthedocs.io/>.

2 PRELIMINARIES AND RELATED WORK

In order to position our contribution and highlight its significance we introduce some important concepts about spatiotemporal data and discuss related literature about geometric deep learning and machine learning software.

2.1 Temporal Graph Sequences

Our framework considers specific input data types on which the spatiotemporal machine learning models operate. Input data types

can differ in terms of the dynamics of the graph and that of the modelled vertex attributes. We take a discrete temporal snapshot view of this data representation problem [25, 26] and our work considers three spatiotemporal data types which can be described by the subplots of Figure 1 and the following formal definitions:

Definition 2.1. Dynamic graph with temporal signal. A dynamic graph with a temporal signal is the ordered set of graph and node feature matrix tuples $\mathcal{D} = \{(\mathcal{G}_1, \mathbf{X}_1), \dots, (\mathcal{G}_T, \mathbf{X}_T)\}$ where the vertex sets satisfy that $V_t = V$, $\forall t \in \{1, \dots, T\}$ and the node feature matrices that $\mathbf{X}_t \in \mathbb{R}^{|V| \times d}$, $\forall t \in \{1, \dots, T\}$.

Definition 2.2. Dynamic graph with static signal. A dynamic graph with a static signal is the ordered set of graph and node feature matrix tuples $\mathcal{D} = \{(\mathcal{G}_1, \mathbf{X}), \dots, (\mathcal{G}_T, \mathbf{X})\}$ where vertex sets satisfy $V_t = V$, $\forall t \in \{1, \dots, T\}$ and the node feature matrix that $\mathbf{X} \in \mathbb{R}^{|V| \times d}$.

Definition 2.3. Static graph with temporal signal. A static graph with a temporal signal is the ordered set of graph and node feature matrix tuples $\mathcal{D} = \{(\mathcal{G}, \mathbf{X}_1), \dots, (\mathcal{G}, \mathbf{X}_T)\}$ where the node feature matrix satisfies that $\mathbf{X}_t \in \mathbb{R}^{|V| \times d}$, $\forall t \in \{1, \dots, T\}$.

Representing spatiotemporal data based on these theoretical concepts allows us the creation of memory efficient data structures which conceptualize these definitions in practice well.

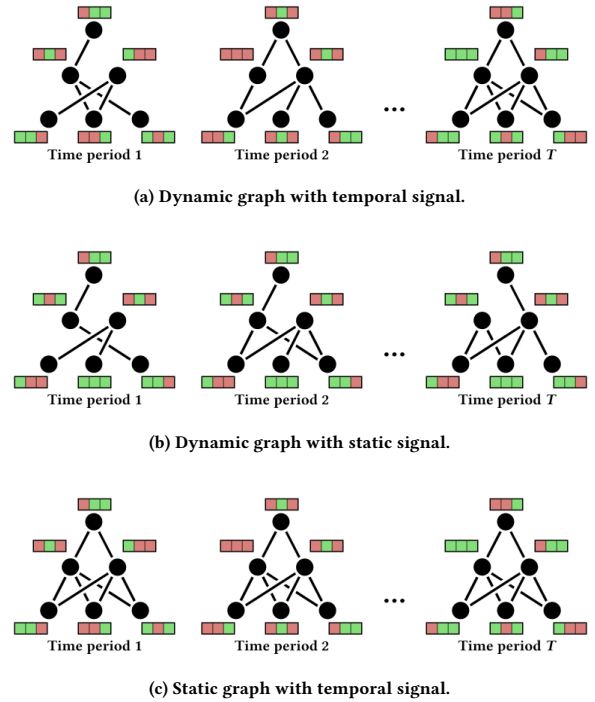


Figure 1: The data iterators in PyTorch Geometric Temporal can provide temporal snapshots for all of the non static geometric deep learning scenarios.

2.2 Deep Learning with Time and Geometry

Our work provides deep learning models that operate on data which has both temporal and spatial aspects. These techniques are natural recombinations of existing neural network layers that operate on sequences and static graph-structured data.

2.2.1 Temporal Deep Learning. A large family of temporal deep learning models such as the *LSTM* [24] and *GRU* [12] generates in-memory representations of data points which are iteratively updated as it learns by new snapshots. Another family of deep learning models uses the *attention mechanism* [3, 35, 59] to learn representations of the data points which are adaptively recontextualized based on the temporal history. These types of models serve as templates for the temporal block of spatiotemporal deep learning models.

2.2.2 Static Graph Representation Learning. Learning representations of vertices, edges and whole graphs with graph neural networks in a supervised or unsupervised way can be described by the *message passing* formalism [17]. In this conceptual framework using the node and edge attributes in a graph as parametric function generates compressed representations (messages) which are propagated between the nodes based on a message-passing rule and aggregated to form new representations. Most of the existing graph neural network architectures such as GCN [30], GGCN[33], ChebyConv [14], and RGCN [50] fit perfectly into this general description of graph neural networks. Models are differentiated by assumptions about the input graph (e.g. node heterogeneity, multiplexity, presence of edge attributes), the message compression function used, the propagation scheme and message aggregation function applied to the received messages.

2.2.3 Spatiotemporal Deep Learning. A spatiotemporal deep learning model fuses the basic conceptual ideas of temporal deep learning techniques and graph representation learning. Operating on a temporal graph sequence these models perform message-passing at each time point with a graph neural network block and the new temporal information is incorporated by a temporal deep learning block. This design allows for sharing salient temporal and spatial autocorrelation information across the spatial units. The temporal and spatial layers which are fused together in a single parametric machine learning model are trained together jointly by exploiting the fact that the fused models are end-to-end differentiable. In Table 1 we summarized the spatiotemporal deep learning models implemented in the framework which we categorized based on the temporal and graph neural network layer blocks, the order of spatial proximity and heterogeneity of the edge set.

2.3 Graph Representation Learning Software

The current graph representation learning software ecosystem which allows academic research and industrial deployment extends open-source auto-differentiation libraries such as TensorFlow [1], PyTorch [41], MxNet [11] and JAX [16, 28]. Our work does the same as we build on the PyTorch Geometric ecosystem. We summarized the characteristics of these libraries in Table 2 which allows for comparing frameworks based on the backend, presence of supervised training functionalities, presence of temporal models and GPU support. Our proposed framework is the only one to date which allows

Table 1: A comparison of spatiotemporal deep learning models in PyTorch Geometric Temporal based on the temporal and spatial block, order of proximity considered and the heterogeneity of the edges.

Model	Temporal Layer	GNN Layer	Proximity Order	Multi Type
DCRNN [32]	GRU	DiffConv	Higher	False
GConvGRU [51]	GRU	Chebyshev	Lower	False
GConvLSTM [51]	LSTM	Chebyshev	Lower	False
GC-LSTM [10]	LSTM	Chebyshev	Lower	True
DyGrAE [54, 55]	LSTM	GGCN	Higher	False
LRGCN [31]	LSTM	RGCN	Lower	False
EGCN-H [39]	GRU	GCN	Lower	False
EGCN-O [39]	LSTM	GCN	Lower	False
T-GCN [65]	GRU	GCN	Lower	False
A3T-GCN [68]	GRU	GCN	Lower	False
AGCRN [4]	GRU	Chebyshev	Higher	False
MPNN LSTM [38]	LSTM	GCN	Lower	False
STGCN [63]	Attention	Chebyshev	Higher	False
ASTGCN [22]	Attention	Chebyshev	Higher	False
MSTGCN [22]	Attention	Chebyshev	Higher	False
GMAN [66]	Attention	Custom	Lower	False
MTGNN [61]	Attention	Custom	Higher	False
AAGCN [52]	Attention	Custom	Higher	False

the supervised training of temporal graph representation learning models with graphics card based acceleration.

Table 2: A desiderata and automatic differentiation backend library based comparison of open-source geometric deep learning libraries.

Library	Backend	Supervised	Temporal	GPU
PT Geometric [15]	PT	✓	✗	✓
Geometric2DR [49]	PT	✗	✗	✓
CogDL [9]	PT	✓	✗	✓
Spektral [21]	TF	✓	✗	✓
TF Geometric [27]	TF	✓	✗	✓
StellarGraph [13]	TF	✓	✗	✓
DGL [67]	TF/PT/MX	✓	✗	✓
DIG [34]	PT	✓	✗	✓
Jraph [18]	JAX	✓	✗	✓
Graph-Learn [62]	Custom	✓	✗	✓
GEM [19]	TF	✗	✗	✓
DynamicGEM [20]	TF	✗	✓	✓
OpenNE [57]	Custom	✗	✗	✗
Karate Club [46]	Custom	✗	✗	✗
Our Work	PT	✓	✓	✓

2.4 Spatiotemporal Data Analytics Software

The open-source ecosystem for spatiotemporal data processing consists of specialized database systems, basic analytical tools and

advanced machine learning libraries. We summarized the characteristics of the most popular libraries in Table 3 with respect to the year of release, purpose of the framework, source code language and GPU support.

First, it is evident that most spatiotemporal data processing tools are fairly new and there is much space for contributions in each subcategory. Second, the database systems are written in high-performance languages while the analytics and machine learning oriented tools have a pure Python/R design or a wrapper written in these languages. Finally, the use of GPU acceleration is not widespread which alludes to the fact that current spatiotemporal data processing tools might have a scalability issue. Our proposed framework PyTorch Geometric Temporal is the first fully open-source GPU accelerated spatiotemporal machine learning library.

Table 3: A multi-aspect comparison of open-source spatiotemporal database systems, data analytics libraries and machine learning frameworks.

Library	Year	Purpose	Language	GPU
GeoWave [60]	2016	Database	Java	✗
StacSpec [23]	2017	Database	Javascript	✗
MobilityDB [69]	2019	Database	C	✗
PyStac [44]	2020	Database	Python	✗
StaRs [42]	2017	Analytics	R	✗
CuSpatial [56]	2019	Analytics	Python	✓
PySAL [43]	2017	Machine Learning	Python	✗
STDMTMB [2]	2018	Machine Learning	R	✗
Our work	2021	Machine Learning	Python	✓

3 THE FRAMEWORK DESIGN

Our primary goal is to give a general theoretical overview of the framework, discuss the framework design choices, give a detailed practical example and highlight our strategy for the long term viability and maintenance of the project.

3.1 Neural Network Layer Design

The spatiotemporal neural network layers are implemented as classes in the framework. Each of the classes has a similar architecture driven by a few simple design principles.

3.1.1 Non-proliferation of classes. The framework reuses the existing high level neural network layer classes as building blocks from the PyTorch and PyTorch Geometric ecosystems. The goal of the library is not to replace the existing frameworks. This design strategy makes sure that the number of auxiliary classes in the framework is kept low and that the framework interfaces well with the rest of the ecosystem.

3.1.2 Hyperparameter inspection and type hinting. The neural network layers do not have default hyperparameter settings as some of these have to be set in a dataset dependent manner. In order to help with this, the layer hyperparameters are stored as public class attributes and they are available for inspection. Moreover, the constructors of the neural network layers use type hinting which helps the end-users to set the hyperparameters.

3.1.3 Limited number of public methods. The spatiotemporal neural network layers in our framework have a limited number of public methods for simplicity. For example, the auxiliary layer initialization methods and other internal model mechanics are implemented as private methods. All of the layers provide a forward method and those which explicitly use the *message-passing* scheme in PyTorch Geometric provide a public *message* method.

3.1.4 Auxiliary layers. The auxiliary neural network layers which are not part of the *PyTorch Geometric* ecosystem such as diffusion convolutional graph neural networks [32] are implemented as standalone neural network layers in the framework. These layers are available for the design of novel neural network architectures as individual components.

3.2 Data Structures

The design of *PyTorch Geometric Temporal* required the introduction of custom data structures which can efficiently store the datasets and provide temporally ordered snapshots for batching.

3.2.1 Spatiotemporal Signal Iterators. Based on the categorization of spatiotemporal signals discussed in Section 2 we implemented three types of *Spatiotemporal Signal Iterators*. These iterators store spatiotemporal datasets in memory efficiently without redundancy. For example a *Static Graph Temporal Signal* iterator will not store the edge indices and weights for each time period in order to save memory. By iterating over a *Spatiotemporal Signal Iterator* at each step a graph snapshot is returned which describes the graph of interest at a given point in time. Graph snapshots are returned in temporal order by the iterators. The *Spatiotemporal Signal Iterators* can be indexed directly to access a specific graph snapshot – a design choice which allows the use of advanced temporal batching.

3.2.2 Graph Snapshots. The time period specific snapshots which consist of labels, features, edge indices and weights are stored as *NumPy* arrays [58] in memory, but returned as a *PyTorch Geometric Data* object instance [15] by the *Spatiotemporal Signal Iterators* when these are iterated on. This design choice hedges against the proliferation of classes and exploits the existing and widely used compact data structures from the *PyTorch* ecosystem [40].

3.2.3 Train-Test Splitting. As part of the library we provide a temporal train-test splitting function which creates train and test snapshot iterators from a *Spatiotemporal Signal Iterator* given a test dataset ratio. This parameter of the splitting function decides the fraction of data that is separated from the end of the spatiotemporal graph snapshot sequence for testing. The returned iterators have the same type as the input iterator. Importantly, this splitting does not influence the applicability of widely used semi-supervised model training strategies such as node masking.

3.2.4 Integrated Benchmark Dataset Loaders. We provided easy-to-use practical data loader classes for widely used existing [38] and the newly released benchmark datasets. These loaders return *Spatiotemporal Signal Iterators* which can be used for training existing and custom designed spatiotemporal neural network architectures to solve supervised machine learning problems.

3.3 Design in Practice Case Study: Cumulative Model Training on CPU

In the following we overview a simple end-to-end machine learning pipeline designed with PyTorch Geometric Temporal. These code snippets solve a practical epidemiological forecasting problem – predicting the weekly number of chickenpox cases in Hungary [47]. The pipeline consists of data preparation, model definition, training and evaluation phases.

```

1 from torch_geometric_temporal import ChickenpoxDatasetLoader
2 from torch_geometric_temporal import temporal_signal_split
3
4 loader = ChickenpoxDatasetLoader()
5
6 dataset = loader.get_dataset()
7
8 train, test = temporal_signal_split(dataset,
9                                   train_ratio=0.9)

```

Listings 1: Loading a default benchmark dataset and creating a temporal split with PyTorch Geometric Temporal

3.3.1 Dataset Loading and Splitting. In Listings 1 as a first step we import the Hungarian chickenpox cases benchmark dataset loader and the temporal train test splitter function (lines 1-2). We define the dataset loader (line 4) and use the `get_dataset()` class method to return a temporal signal iterator (line 5). Finally, we create a train-test split of the spatiotemporal dataset by using the splitting function and retain 10% of the temporal snapshots for model performance evaluation (lines 7-8).

```

1 import torch
2 import torch.nn.functional as F
3 from torch_geometric_temporal.nn.recurrent import DCRNN
4
5 class RecurrentGCN(torch.nn.Module):
6     def __init__(self, node_features, filters):
7         super(RecurrentGCN, self).__init__()
8         self.recurrent = DCRNN(node_features, filters, 1)
9         self.linear = torch.nn.Linear(filters, 1)
10
11     def forward(self, x, edge_index, edge_weight):
12         h = self.recurrent(x, edge_index, edge_weight)
13         h = F.relu(h)
14         h = F.dropout(h, training=self.training)
15         h = self.linear(h)
16         return h

```

Listings 2: Defining a recurrent graph convolutional neural network using PyTorch Geometric Temporal consisting of a diffusion convolutional spatiotemporal layer followed by rectified linear unit activations, dropout and a feedforward neural network layer.

3.3.2 Recurrent Graph Convolutional Model Definition. We define a recurrent graph convolutional neural network model in Listings 2. We import the base and functional programming PyTorch libraries and one of the neural network layers from PyTorch Geometric Temporal (lines 1-3). The model requires a node feature count and

convolutional filter parameter in the constructor (line 6). The model consists of a one-hop Diffusion Convolutional Recurrent Neural Network layer [32] and a fully connected layer with a single neuron (lines 8-9).

In the forward pass method of the neural network the model uses the vertex features, edges and the optional edge weights (line 11). The initial recurrent graph convolution based aggregation (line 12) is followed by a rectified linear unit activation function [37] and dropout [53] for regularization (lines 13-14). Using the fully-connected layer the model outputs a single score for each spatial unit (lines 15-16).

```

1 model = RecurrentGCN(node_features=8, filters=32)
2
3 optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
4
5 model.train()
6
7 for epoch in range(200):
8     cost = 0
9     for time, snapshot in enumerate(train):
10         y_hat = model(snapshot.x,
11                       snapshot.edge_index,
12                       snapshot.edge_attr)
13         cost = cost + torch.mean((y_hat-snapshot.y)**2)
14     cost = cost / (time+1)
15     cost.backward()
16     optimizer.step()
17     optimizer.zero_grad()

```

Listings 3: Creating a recurrent graph convolutional neural network instance and training it by cumulative weight updates.

3.3.3 Model Training. Using the dataset split and the model definition we can turn our attention to training a regressor. In Listings 3 we create a model instance (line 1), transfer the model parameters (line 3) to the Adam optimizer [29] which uses a learning rate of 0.01 and set the model to be trainable (line 5). In each epoch we set the accumulated cost to be zero (line 8) iterate over the temporal snapshots in the training data (line 9), make forward passes with the model on each temporal snapshot and accumulate the spatial unit specific mean squared errors (lines 10-13). We normalize the cost, backpropagate and update the model parameters (lines 14-17).

```

1 model.eval()
2 cost = 0
3 for time, snapshot in enumerate(test):
4     y_hat = model(snapshot.x,
5                   snapshot.edge_index,
6                   snapshot.edge_attr)
7     cost = cost + torch.mean((y_hat-snapshot.y)**2)
8 cost = cost / (time+1)
9 cost = cost.item()
10 print("MSE: {:.4f}".format(cost))

```

Listings 4: Evaluating the recurrent graph convolutional neural network on the test portion of the spatiotemporal dataset using the time unit averaged mean squared error.

3.3.4 Model Evaluation. The scoring of the trained recurrent graph neural network in Listings 4 uses the snapshots in the test dataset. We set the model to be non trainable and the accumulated squared error as zero (lines 1-2). We iterate over the test spatiotemporal snapshots, make forward passes to predict the number of chickenpox cases and accumulate the squared error (lines 3-7). The accumulated errors are normalized and we can print the mean squared error calculated on the whole test horizon (lines 8-10).

3.4 Design in Practice Case Study: Incremental Model Training with GPU Acceleration

Exploiting the power of GPU based acceleration of computations happens at the training and evaluation steps of the PyTorch Geometric Temporal pipelines. In this case study we assume that the Hungarian Chickenpox cases dataset is already loaded in memory, the temporal split happened and a model class was defined by the code snippets in Listings 1 and 2. Moreover, we assume that the machine used for training the neural network can access a single CUDA compatible GPU device [48].

```

1 model = RecurrentGCN(node_features=8, filters=32)
2 device = torch.device('cuda')
3 model = model.to(device)
4
5 optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
6 model.train()
7
8 for epoch in range(200):
9     for snapshot in train:
10         snapshot = snapshot.to(device)
11         y_hat = model(snapshot.x,
12                        snapshot.edge_index,
13                        snapshot.edge_attr)
14         cost = torch.mean((y_hat-snapshot.y)**2)
15         cost.backward()
16         optimizer.step()
17         optimizer.zero_grad()

```

Listings 5: Creating a recurrent graph convolutional neural network instance and training it by incremental weight updates on a GPU.

3.4.1 Model Training. In Listings 5 we demonstrate accelerated training with incremental weight updates. The model of interest and the device used for training are defined while the model is transferred to the GPU (lines 1-3). The optimizer registers the model parameters and the model parameters are set to be trainable (lines 5-6). We iterate over the temporal snapshot iterator 200 times and the iterator returns a temporal snapshot in each step. Importantly the snapshots which are PyTorch Geometric Data objects are transferred to the GPU (lines 8-10). The use of PyTorch Geometric Data objects as temporal snapshots allows the transfer of the time period specific edges, node features and target vector with a single command. Using the input data a forward pass is made, loss is accumulated and weight updates happen using the optimizer in each time period (lines 11-17). Compared to the cumulative backpropagation based training approach discussed in Subsection 3.3

this backpropagation strategy is slower as weight updates happen at each time step, not just at the end of training epochs.

3.4.2 Model Evaluation. During model scoring the GPU can be utilized again. The snippet in Listings 6 demonstrates that the only modification needed for accelerated evaluation is the transfer of snapshots to the GPU. In each time period we move the temporal snapshot to the device to do the forward pass (line 4). We do the forward pass with the model and the snapshot on the GPU and accumulate the loss (lines 5-8). The loss value is averaged out and detached from the GPU for printing (lines 9-11).

```

1 model.eval()
2 cost = 0
3 for time, snapshot in enumerate(test):
4     snapshot = snapshot.to(device)
5     y_hat = model(snapshot.x,
6                  snapshot.edge_index,
7                  snapshot.edge_attr)
8     cost = cost + torch.mean((y_hat-snapshot.y)**2)
9 cost = cost / (time+1)
10 cost = cost.item()
11 print("MSE: {:.4f}".format(cost))

```

Listings 6: Evaluating the recurrent graph convolutional neural network with GPU based acceleration.

3.5 Maintaining PyTorch Geometric Temporal

The viability of the project is made possible by the open-source code, version control, public releases, automatically generated documentation, continuous integration, and near 100% test coverage.

3.5.1 Open-Source Code-Base and Public Releases. The source code of *PyTorch Geometric Temporal* is publicly available on *GitHub* under the MIT license. Using an open version control system allowed us to have a large group collaborate on the project and have external contributors who also submitted feature requests. The public releases of the library are also made available on the *Python Package Index*, which means that the framework can be installed via the *pip* command using the terminal.

3.5.2 Documentation. The source-code of *PyTorch Geometric Temporal* and *Sphinx* [8] are used to generate a publicly available documentation of the library at <https://pytorch-geometric-temporal.readthedocs.io/>. This documentation is automatically created every time when the code-base changes in the public repository. The documentation covers the constructors and public methods of neural network layers, temporal signal iterators, public dataset loaders and splitters. It also includes a list of relevant research papers, an in-depth installation guide, a detailed getting-started tutorial and a list of integrated benchmark datasets.

3.5.3 Continuous Integration. We provide continuous integration for *PyTorch Geometric Temporal* with *GitHub Actions* which are available for free on *GitHub* without limitations on the number of builds. When the code is updated on any branch of the repository the build process is triggered and the library is deployed on *Linux*, *Windows* and *macOS* virtual machines.

3.5.4 Unit Tests and Code Coverage. The temporal graph neural network layers, custom data structures and benchmark dataset loaders are all covered by unit tests. These unit tests can be executed locally using the source code. Unit tests are also triggered by the continuous integration provided by *GitHub Actions*. When the master branch of the open-source *GitHub* repository is updated, the build is successful, and all of the unit tests pass a coverage report is generated by *CodeCov*.

4 EXPERIMENTAL EVALUATION

The proposed framework is evaluated on node level regression tasks using novel datasets which we release with the paper. We also evaluate the effect of various batching techniques on the predictive performance and runtime.

4.1 Datasets

We release new spatiotemporal benchmark datasets with *PyTorch Geometric Temporal* which can be used to test models on node level regression tasks. The descriptive statistics and properties of these newly introduced benchmark datasets are summarized in Table 4.

Table 4: Properties and granularity of the spatiotemporal datasets introduced in the paper with information about the number of time periods (T) and spatial units ($|V|$).

Dataset	Signal	Graph	Frequency	T	$ V $
Chickenpox Hungary	Temporal	Static	Weekly	522	20
Windmill Large	Temporal	Static	Hourly	17,472	319
Windmill Medium	Temporal	Static	Hourly	17,472	26
Windmill Small	Temporal	Static	Hourly	17,472	11
Pedal Me Deliveries	Temporal	Static	Weekly	36	15
Wikipedia Math	Temporal	Static	Daily	731	1,068
Twitter Tennis RG	Static	Dynamic	Hourly	120	1000
Twitter Tennis UO	Static	Dynamic	Hourly	112	1000
Covid19 England	Temporal	Dynamic	Daily	61	129
Montevideo Buses	Temporal	Static	Hourly	744	675
MTM-1 Hand Motions	Temporal	Static	1/24 Seconds	14,469	21

These newly released datasets are the following:

- **Chickenpox Hungary.** A spatiotemporal dataset about the officially reported cases of chickenpox in Hungary. The nodes are counties and edges describe direct neighbourhood relationships. The dataset covers the weeks between 2005 and 2015 without missingness.
- **Windmill Output Datasets.** An hourly windfarm energy output dataset covering 2 years from a European country. Edge weights are calculated from the proximity of the windmills – high weights imply that two windmill stations are in close vicinity. The size of the dataset relates to the grouping of windfarms considered; the smaller datasets are more localized to a single region.
- **Pedal Me Deliveries.** A dataset about the number of weekly bicycle package deliveries by Pedal Me in London during 2020 and 2021. Nodes in the graph represent geographical units and edges are proximity based mutual adjacency relationships.
- **Wikipedia Math.** Contains Wikipedia pages about popular mathematics topics and edges describe the links from one page to another. Features describe the number of daily visits between 2019 and 2021 March.
- **Twitter Tennis RG and UO.** Twitter mention graphs of major tennis tournaments from 2017. Each snapshot contains the graph of popular player or sport news accounts and mentions between them [5, 6]. Node labels encode the number of mentions received and vertex features are structural properties.
- **Covid19 England.** A dataset about mass mobility between regions in England and the number of confirmed COVID-19 cases from March to May 2020 [38]. Each day contains a different mobility graph and node features corresponding to the number of cases in the previous days. Mobility stems from Facebook Data For Good ¹ and cases from gov.uk ².
- **Montevideo Buses.** A dataset about the hourly passenger inflow at bus stop level for eleven bus lines from the city of Montevideo. Nodes are bus stops and edges represent connections between the stops; the dataset covers a whole month of traffic patterns.
- **MTM-1 Hand Motions.** A temporal dataset of Methods-Time Measurement-1 [36] motions, signalled as consecutive graph frames of 21 3D hand key points that were acquired via MediaPipe Hands [64] from original RGB-Video material. Node features encode the normalized xyz-coordinates of each finger joint and the vertices are connected according to the human hand structure.

4.2 Predictive Performance

The forecasting experiments focus on the evaluation of the recurrent graph neural networks implemented in our framework. We compare the predictive performance under two specific backpropagation regimes which can be used to train these recurrent models:

- **Incremental:** After each temporal snapshot the loss is backpropagated and model weights are updated. This would need as many weight updates as the number of temporal snapshots.
- **Cumulative:** When the loss from every temporal snapshot is aggregated it is backpropagated and weights are updated with the optimizer. This requires one weight update per epoch.

4.2.1 Experimental settings. Using 90% of the temporal snapshots for training, we evaluated the forecasting performance on the last 10% by calculating the average mean squared error from 10 experimental runs. We used models with a recurrent graph convolutional layer which had 32 convolutional filters. The spatiotemporal layer was followed by the rectified linear unit [37] activation function and during training time we used a dropout of 0.5 for regularization [53] after the spatiotemporal layer. The hidden representations were fed to a fully connected feedforward layer which outputted the predicted scores for each spatial unit. The recurrent models

¹<https://dataforgood.fb.com/>

²<https://coronavirus.data.gov.uk/>

Table 5: The predictive performance of spatiotemporal neural networks evaluated by average mean squared error. We report average performances calculated from 10 experimental repetitions with standard deviations around the average mean squared error calculated on 10% forecasting horizons. We use the incremental and cumulative backpropagation strategies.

	Chickenpox Hungary		Twitter Tennis RG		PedalMe London		Wikipedia Math	
	Incremental	Cumulative	Incremental	Cumulative	Incremental	Cumulative	Incremental	Cumulative
DCRNN [32]	1.124 $\pm$ 0.015	1.123 $\pm$ 0.014	2.049 $\pm$ 0.023	2.043 $\pm$ 0.016	1.463 $\pm$ 0.019	1.450 $\pm$ 0.024	0.679 $\pm$ 0.020	0.803 $\pm$ 0.018
GConvGRU [51]	1.128 $\pm$ 0.011	1.132 $\pm$ 0.023	2.051 $\pm$ 0.020	2.007 $\pm$ 0.022	1.622 $\pm$ 0.032	1.944 $\pm$ 0.013	0.657 $\pm$ 0.015	0.837 $\pm$ 0.021
GConvLSTM [51]	1.121 $\pm$ 0.014	1.119 $\pm$ 0.022	2.049 $\pm$ 0.024	2.007 $\pm$ 0.012	1.442 $\pm$ 0.028	1.433 $\pm$ 0.020	0.777 $\pm$ 0.021	0.868 $\pm$ 0.018
GC-LSTM [10]	1.115 $\pm$ 0.014	1.116 $\pm$ 0.023	2.053 $\pm$ 0.024	2.032 $\pm$ 0.015	1.455 $\pm$ 0.023	1.468 $\pm$ 0.025	0.779 $\pm$ 0.023	0.852 $\pm$ 0.016
DyGrAE [54, 55]	1.120 $\pm$ 0.021	1.118 $\pm$ 0.015	2.031 $\pm$ 0.006	2.007 $\pm$ 0.004	1.455 $\pm$ 0.031	1.456 $\pm$ 0.019	0.773 $\pm$ 0.009	0.816 $\pm$ 0.016
EGCN-H [39]	1.113 $\pm$ 0.016	1.104 $\pm$ 0.024	2.040 $\pm$ 0.018	2.006 $\pm$ 0.008	1.467 $\pm$ 0.026	1.436 $\pm$ 0.017	0.775 $\pm$ 0.022	0.857 $\pm$ 0.022
EGCN-O [39]	1.124 $\pm$ 0.009	1.119 $\pm$ 0.020	2.055 $\pm$ 0.020	2.010 $\pm$ 0.014	1.491 $\pm$ 0.024	1.430 $\pm$ 0.023	0.750 $\pm$ 0.014	0.823 $\pm$ 0.014
A3T-GCN [68]	1.114 $\pm$ 0.008	1.119 $\pm$ 0.018	2.045 $\pm$ 0.021	2.008 $\pm$ 0.016	1.469 $\pm$ 0.027	1.475 $\pm$ 0.029	0.781 $\pm$ 0.011	0.872 $\pm$ 0.017
T-GCN [65]	1.117 $\pm$ 0.011	1.111 $\pm$ 0.022	2.045 $\pm$ 0.027	2.008 $\pm$ 0.017	1.479 $\pm$ 0.012	1.481 $\pm$ 0.029	0.764 $\pm$ 0.011	0.846 $\pm$ 0.020
MPNN LSTM [38]	1.116 $\pm$ 0.023	1.129 $\pm$ 0.021	2.053 $\pm$ 0.041	2.007 $\pm$ 0.010	1.485 $\pm$ 0.028	1.458 $\pm$ 0.013	0.795 $\pm$ 0.010	0.905 $\pm$ 0.017
AGCRN [4]	1.120 $\pm$ 0.010	1.116 $\pm$ 0.017	2.039 $\pm$ 0.022	2.010 $\pm$ 0.009	1.469 $\pm$ 0.030	1.465 $\pm$ 0.026	0.788 $\pm$ 0.011	0.832 $\pm$ 0.020

were trained for 100 epochs with the Adam optimizer [29] which used a learning rate of 10^{-2} to minimize the mean squared error.

4.2.2 Experimental findings. Results are presented in Table 5 where we also report standard deviations around the test set mean squared error and bold numbers denote the best performing model under each training regime on a dataset. Our experimental findings demonstrate multiple important empirical regularities which have important practical implications. Namely these are the following:

- (1) Most recurrent graph neural networks have a similar predictive performance on these regression tasks. In simple terms there is not a single model which acts as *silver bullet*. This also postulates that the model with the lowest training time is likely to be as good as the slowest one.
- (2) Results on the Wikipedia Math dataset imply that a cumulative backpropagation strategy can have a detrimental effect on the predictive performance of a recurrent graph neural network. When computation resources are not a bottleneck an incremental strategy can be significantly better.

4.3 Runtime Performance

The evaluation of the PyTorch Geometric Temporal runtime performance focuses on manipulating the input size and measuring the time needed to complete a training epoch. We investigate the runtime under the incremental and cumulative backpropagation strategies.

4.3.1 Experimental settings. The runtime evaluation used the GConvGRU model [51] with the hyperparameter settings described in Subsection 4.2. We measured the time needed for doing a single epoch over a sequence of 100 synthetic graphs. Reference Watts-Strogatz graphs in the snapshots of the dynamic graph with temporal signal iterator had binary labels, 2^{10} nodes, 2^5 edges per node and 2^5 node features. Runtimes were measured on the following hardware:

- **CPU:** The machine used for benchmarking had 8 *Intel i5-1035G1* processors.
- **GPU:** We utilized a machine with a single *Tesla V-100* graphics card for the experiments.

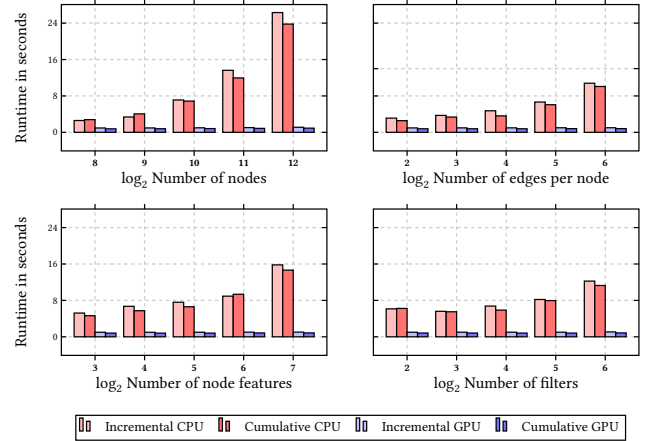


Figure 2: The average time needed for doing an epoch on a dynamic graph – temporal signal iterator of Watts Strogatz graphs with a recurrent graph convolutional model.

4.3.2 Experimental findings. We plotted the average runtime calculated from 10 experimental runs on Figure 2 for each input size. Our results about runtime have two important implications about the practical application of our framework:

- (1) The use of a cumulative backpropagation strategy only results in marginal computation gains compared to the incremental one.
- (2) On temporal sequences of large dynamically changing graphs the GPU aided training can reduce the time needed to do an epoch by a whole magnitude.

5 CONCLUSIONS AND FUTURE DIRECTIONS

In this paper we discussed *PyTorch Geometric Temporal* the first deep learning library designed for neural spatiotemporal signal processing. We reviewed the existing geometric deep learning and machine learning techniques implemented in the framework. We gave an overview of the general machine learning framework design principles, the newly introduced input and output data structures, long-term project viability and discussed a case study with source-code which utilized the library. Our empirical evaluation focused on (a) the predictive performance of the models available in the library on real world datasets which we released with the framework; (b) the scalability of the methods under various input sizes and structures.

Our work could be extended and it also opens up opportunities for novel geometric deep learning and applied machine learning research. A possible direction to extend our work would be the consideration of continuous time or time differences between temporal snapshots which are not constant. Another opportunity is the inclusion of temporal models which operate on curved spaces such as hyperbolic and spherical spaces. We are particularly interested in how the spatiotemporal deep learning techniques in the framework can be deployed and used for solving high-impact practical machine learning tasks.

REFERENCES

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. Tensorflow: A System for Large-Scale Machine Learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)*. 265–283.
- [2] Sean Anderson, Eric Ward, Lewis Barnett, and Philippina English. 2018. sdmTMB: Spatial and spatiotemporal GLMMs with TMB. <https://github.com/pbs-assess/sdmTMB>.
- [3] Dzmitry Bahdanau, Kyung Hyun Cho, and Yoshua Bengio. 2015. Neural Machine Translation by Jointly Learning to Align and Translate. In *3rd International Conference on Learning Representations, ICLR 2015*.
- [4] Lei Bai, Lina Yao, Can Li, Xianzhi Wang, and Can Wang. 2020. Adaptive Graph Convolutional Recurrent Network for Traffic Forecasting. *Advances in Neural Information Processing Systems* 33 (2020).
- [5] Ferenc B eres, Domokos M. Kelen, R obert P alovics, and Andr as A. Bencz ur. 2019. Node Embeddings in Dynamic Graphs. *Applied Network Science* 4, 64 (2019), 25.
- [6] Ferenc B eres, R obert P alovics, Anna Ol ah, and Andr as A. Bencz ur. 2018. Temporal Walk Based Centrality Metric for Graph Streams. *Applied Network Science* 3, 32 (2018), 26.
- [7] Aleksandar Bojchevski, Johannes Klicpera, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek R ozemberczki, Mich al Lukasik, and Stephan G unnemann. 2020. Scaling Graph Neural Networks with Approximate Pagerank. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2464–2473.
- [8] Georg Brandl. 2010. Sphinx Documentation. URL <http://sphinx-doc.org/sphinx.pdf> (2010).
- [9] Yukuo Cen, Zhenyu Hou, Yan Wang, Qibin Chen, Yizhen Luo, Xingcheng Yao, Aohan Zeng, Shiguang Guo, Peng Zhang, Guohao Dai, et al. 2021. CogDL: An Extensive Toolkit for Deep Learning on Graphs. *arXiv preprint arXiv:2103.00959* (2021).
- [10] Jinyin Chen, Xuanheng Xu, Yangyang Wu, and Haibin Zheng. 2018. GC-LSTM: Graph Convolution Embedded LSTM for Dynamic Link Prediction. *arXiv preprint arXiv:1812.04206* (2018).
- [11] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. MXNet: A Flexible and Efficient Machine Learning Library for Heterogeneous Distributed Systems. *arXiv preprint arXiv:1512.01274* (2015).
- [12] Kyunghyun Cho, Bart van Merri enboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 1724–1734.
- [13] CSIRO’s Data61. 2018. StellarGraph Machine Learning Library. <https://github.com/stellargraph/stellargraph>.
- [14] Micha el Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. In *Advances in Neural Information Processing Systems*. 3844–3852.
- [15] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [16] Roy Frostig, Matthew James Johnson, and Chris Leary. 2018. Compiling Machine Learning Programs via High-Level Tracing. *Systems for Machine Learning* (2018).
- [17] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *International Conference on Machine Learning*. PMLR, 1263–1272.
- [18] Jonathan Godwin*, Thomas Keck*, Peter Battaglia, Victor Bapst, Thomas Kipf, Yujia Li, Kimberly Stachenfeld, Petar Veli kovi , and Alvaro Sanchez-Gonzalez. 2020. *Jraph: A Library for Graph Neural Networks in Jax*. <http://github.com/deepmind/jraph>
- [19] Palash Goyal, Sujit Rokka Chhetri, Ninareh Mehrabi, Emilio Ferrara, and Arquimedes Canedo. 2018. DynamicGEM: A Library for Dynamic Graph Embedding Methods. *arXiv preprint arXiv:1811.10734* (2018).
- [20] Palash Goyal and Emilio Ferrara. [n.d.]. GEM: A Python Package for Graph Embedding Methods. *Journal of Open Source Software* 3, 29 (n.d.), 876.
- [21] Daniele Grattarola and Cesare Alippi. 2020. Graph Neural Networks in TensorFlow and Keras with Spektral. *arXiv preprint arXiv:2006.12138* (2020).
- [22] Shengnan Guo, Youfang Lin, Ning Feng, Chao Song, and Huaiyu Wan. 2019. Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 922–929.
- [23] Matthew Hanson. 2019. The Open-Source Software Ecosystem for Leveraging Public Datasets in Spatio-Temporal Asset Catalogs (STAC). In *AGU Fall Meeting Abstracts*, Vol. 2019. IN23B–07.
- [24] Sepp Hochreiter and J urgen Schmidhuber. 1997. Long Short-Term Memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [25] Petter Holme. 2015. Modern Temporal Network Theory: A Colloquium. *The European Physical Journal B* 88, 9 (2015), 1–30.
- [26] Petter Holme and Jari Saram aki. 2012. Temporal Networks. *Physics reports* 519, 3 (2012), 97–125.
- [27] Jun Hu, Shengsheng Qian, Quan Fang, Youze Wang, Quan Zhao, Huaiwen Zhang, and Changsheng Xu. 2021. Efficient Graph Deep Learning in TensorFlow with TF Geometric. *arXiv preprint arXiv:2101.11552* (2021).
- [28] James Bradbury and Roy Frostig and Peter Hawkins and Matthew James Johnson and Chris Leary and Dougal Maclaurin and George Necula and Adam Paszke and Jake VanderPlas and Skye Wanderman-Milne and Qiao Zhang. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. <http://github.com/google/jax>
- [29] Diederik Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations*.
- [30] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.
- [31] Jia Li, Zhichao Han, Hong Cheng, Jiao Su, Pengyun Wang, Jianfeng Zhang, and Lujia Pan. 2019. Predicting Path Failure in Time-Evolving Graphs. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1279–1289.
- [32] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations*.
- [33] Yujia Li, Richard Zemel, Marc Brockschmidt, and Daniel Tarlow. 2016. Gated Graph Sequence Neural Networks. In *International Conference on Learning Representations (ICLR)*.
- [34] Meng Liu, Youzhi Luo, Limei Wang, Yaochen Xie, Hao Yuan, Shurui Gui, Zhao Xu, Haiyang Yu, Jingtun Zhang, Yi Liu, Keqiang Yan, Bora Oztekin, Haoran Liu, Xuan Zhang, Cong Fu, and Shuiwang Ji. 2021. DiG: A Turnkey Library for Diving into Graph Deep Learning Research. *arXiv preprint arXiv:2103.12608* (2021).
- [35] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. 2015. Effective Approaches to Attention-based Neural Machine Translation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. 1412–1421.
- [36] Harold B Maynard, G J Stegemerten, and John L Schwab. 1948. *Methods-Time Measurement*. McGraw-Hill.
- [37] Vinod Nair and Geoffrey E Hinton. 2010. Rectified Linear Units Improve Restricted Boltzmann Machines. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*. 807–814.
- [38] George Panagopoulos, Giannis Nikolentzos, and Mich alis Vazirgiannis. 2021. Transfer Graph Neural Networks for Pandemic Forecasting. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence*.
- [39] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B Schardl, and Charles E Leiserson. 2020. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. In *AAAI*. 5363–5370.

- [40] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*. 8024–8035.
- [41] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems* 32 (2019), 8026–8037.
- [42] Edzer Pebesma. 2017. staRS: Spatiotemporal Arrays: Raster and Vector Datacubes. <https://github.com/r-spatial/stars>.
- [43] Sergio J Rey and Luc Anselin. 2010. PySAL: A Python Library of Spatial Analytical Methods. In *Handbook of Applied Spatial Analysis*. Springer, 175–193.
- [44] Emanuele Rob. 2020. PySTAC: Python library for working with any SpatioTemporal Asset Catalog (STAC). <https://github.com/stac-utils/pystac>. *GitHub repository*.
- [45] Benedek Rozemberczki, Peter Englert, Amol Kapoor, Martin Blais, and Bryan Perozzi. 2020. Pathfinder Discovery Networks for Neural Message Passing. *arXiv preprint arXiv:2010.12878* (2020).
- [46] Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. 2020. Karate Club: An API Oriented Open-source Python Framework for Unsupervised Learning on Graphs. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 3125–3132.
- [47] Benedek Rozemberczki, Paul Scherer, Oliver Kiss, Rik Sarkar, and Tamas Ferenci. 2021. Chickenpox Cases in Hungary: a Benchmark Dataset for Spatiotemporal Signal Processing with Graph Neural Networks. *arXiv:2102.08100 [cs.LG]*
- [48] Jason Sanders and Edward Kandrot. 2010. *CUDA by Example: An Introduction to General-Purpose GPU Programming*. Addison-Wesley Professional.
- [49] Paul Scherer and Pietro Lio. 2020. Learning Distributed Representations of Graphs with Geo2DR. In *ICML Workshop on Graph Representation Learning and Beyond*.
- [50] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling Relational Data with Graph Convolutional Networks. In *European Semantic Web Conference*. Springer, 593–607.
- [51] Youngjoo Seo, Michaël Defferrard, Pierre Vandergheynst, and Xavier Bresson. 2018. Structured Sequence Modeling with Graph Convolutional Recurrent Networks. In *International Conference on Neural Information Processing*. Springer, 362–373.
- [52] Lei Shi, Yifan Zhang, Jian Cheng, and Hanqing Lu. 2019. Two-Stream Adaptive Graph Convolutional Networks for Skeleton-Based Action Recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12026–12035.
- [53] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *The Journal of Machine Learning Research* 15, 1 (2014), 1929–1958.
- [54] Aynaz Taheri and Tanya Berger-Wolf. 2019. Predictive Temporal Embedding of Dynamic Graphs. In *Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. 57–64.
- [55] Aynaz Taheri, Kevin Gimpel, and Tanya Berger-Wolf. 2019. Learning to Represent the Evolution of Dynamic Graphs with Recurrent Models. In *Companion Proceedings of The 2019 World Wide Web Conference (WWW '19)*. 301–307.
- [56] Paul Taylor, Christopher Harris, Thompson Comer, and Mark Harris. 2019. CUDA-Accelerated GIS and Spatiotemporal Algorithms. <https://github.com/rapidsai/cuspatial>.
- [57] Cunchao Tu, Yuan Yao, Zhengyan Zhang, Ganqu Cui, Hao Wang, Changxin Tian, Jie Zhou, and Cheng Yang. 2018. OpenNE: An Open Source Toolkit for Network Embedding. <https://github.com/thunlp/OpenNE>.
- [58] Stefan Van Der Walt, S Chris Colbert, and Gael Varoquaux. 2011. The NumPy Array: a Structure for Efficient Numerical Computation. *Computing in science & engineering* 13, 2 (2011), 22–30.
- [59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 6000–6010.
- [60] Michael A Whitby, Rich Fecher, and Chris Bennight. 2017. GeoWave: Utilizing Distributed Key-Value Stores for Multidimensional Data. In *International Symposium on Spatial and Temporal Databases*. Springer, 105–122.
- [61] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaoju Chang, and Chengqi Zhang. 2020. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 753–763.
- [62] Hongxia Yang. 2019. AliGraph: A Comprehensive Graph Neural Network Platform. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3165–3166.
- [63] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-Temporal Graph Convolutional Networks: a Deep Learning Framework for Traffic Forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*. 3634–3640.
- [64] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. MediaPipe Hands: On-device Real-time Hand Tracking. *arXiv:2006.10214 [cs.CV]*
- [65] Ling Zhao, Yujiao Song, Chao Zhang, Yu Liu, Pu Wang, Tao Lin, Min Deng, and Haifeng Li. 2019. T-GCN: A Temporal Graph Convolutional Network for Traffic Prediction. *IEEE Transactions on Intelligent Transportation Systems* 21, 9 (2019), 3848–3858.
- [66] Chuanpan Zheng, Xiaoliang Fan, Cheng Wang, and Jianzhong Qi. 2020. GMAN: A Graph Multi-Attention Network for Traffic Prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 1234–1241.
- [67] Da Zheng, Minjie Wang, Quan Gan, Zheng Zhang, and George Karypis. 2020. Learning Graph Neural Networks with Deep Graph Library. In *Companion Proceedings of the Web Conference 2020 (WWW '20)*. 305–306.
- [68] Jiawei Zhu, Yujiao Song, Ling Zhao, and Haifeng Li. 2020. A3T-GCN: Attention Temporal Graph Convolutional Network for Traffic Forecasting. *arXiv preprint arXiv:2006.11583* (2020).
- [69] Esteban Zimányi, Mahmoud Sakr, and Arthur Lesuisse. 2020. MobilityDB: A Mobility Database Based on PostgreSQL and PostGIS. *ACM Transactions on Database Systems (TODS)* 45, 4 (2020), 1–42.